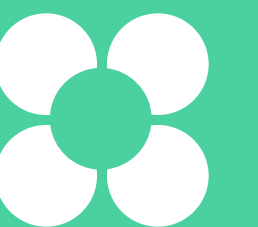
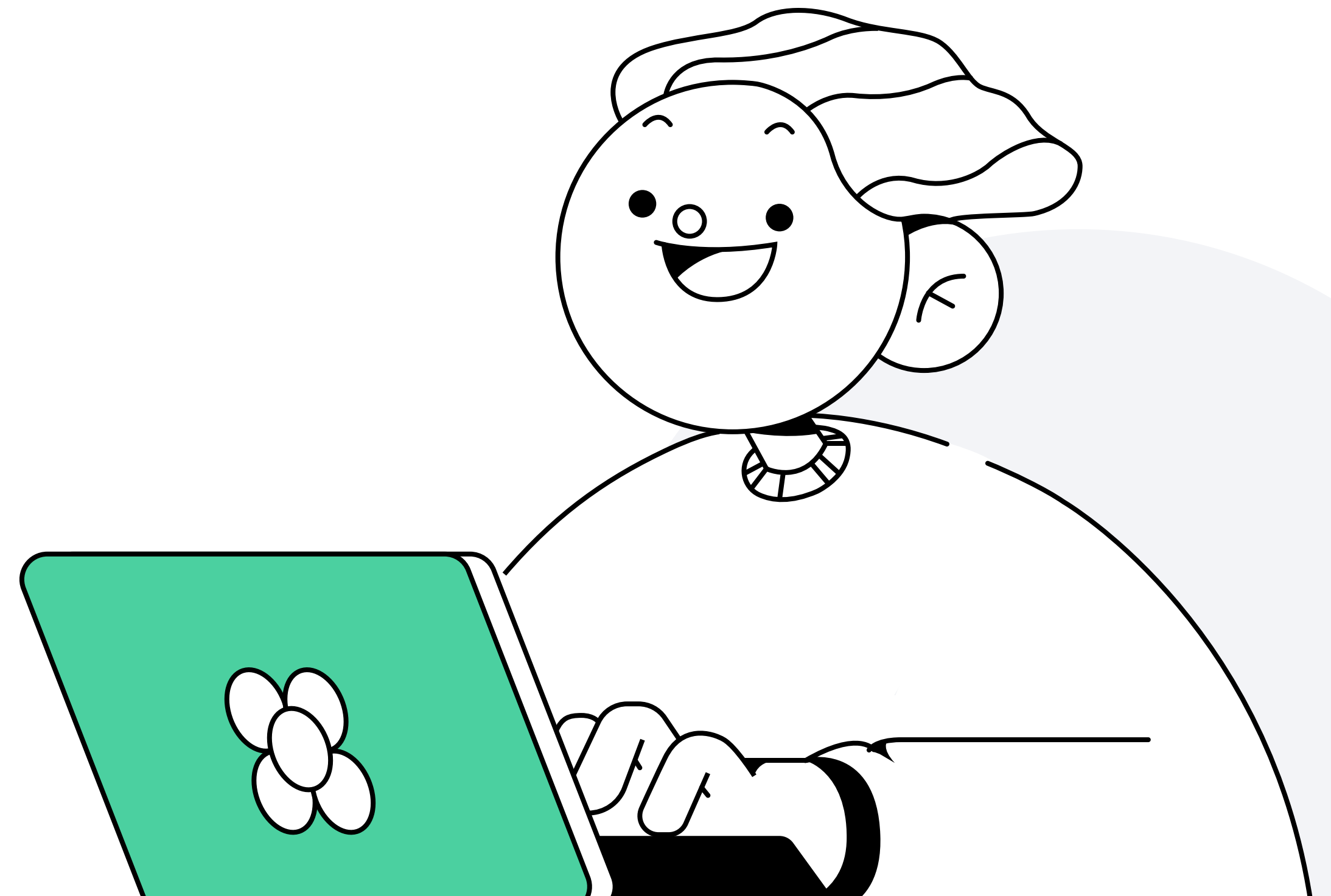


Типы данных Java: объекты



План занятия

- ① Особенности ссылочных типов данных
- ② Тип данных String
- ③ Тип данных Enum

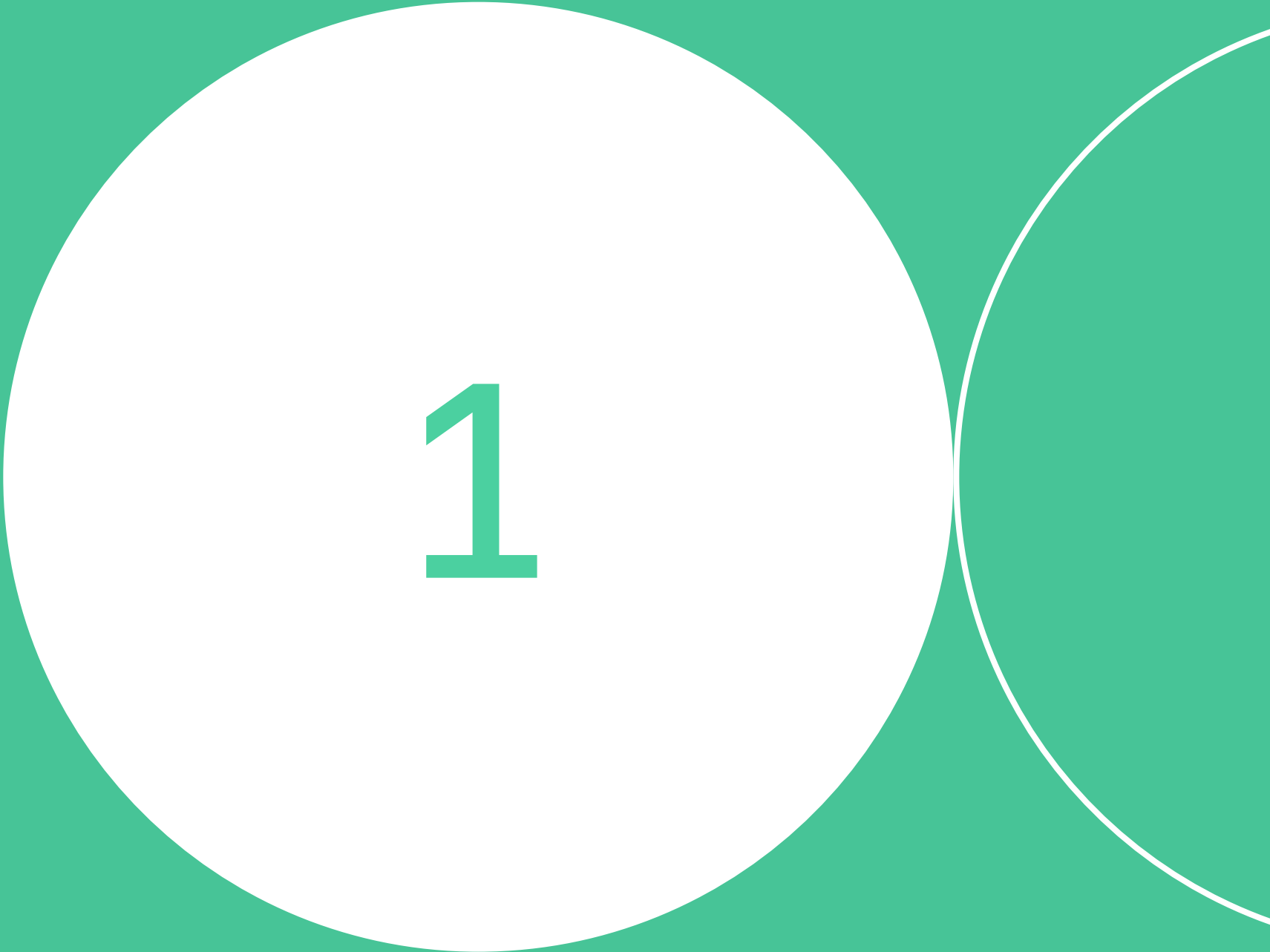


Типы данных в Java

→ Примитивные (primitive data types)

→ **Ссылочные (reference types)**

Особенности ссылочных типов данных



1

Сравнение ссылочных и примитивных типов данных

Параметры сравнения	Ссылочные (reference types)	Примитивные (primitive data types)
Хранение и использование данных	В специальной области памяти — heap — создаётся ячейка. Значение записывается в ячейку в виде бинарного кода. А в переменную идёт ссылка на адрес ячейки в heap	Выделяется ячейка, в неё записывается значение при помощи бинарного кода (зависит от размера ячейки). В переменной записывается само значение
Операции над данными	Используются специальные команды	Используются различные знаки: +, - -, / и т. д.
Способ создания	Создаём объекты через new . Scanner Scanner = new scanner (System.in)	Создаём данные с использованием значков — цифры, буквы, дробные цифры

Пример создания ссылки на объект

```
Object example1 = null;  
String example2 = null;
```

Где:

- **Object** и **String** — тип создаваемой ссылки
- **example1** и **example2** — имена ссылок, по которым мы сможем дальше обращаться к этим объектам в программе
- **null** — ключевое слово в Java, обозначающее отсутствие, то есть фактически **example1** и **example2** — это ссылки на несуществующие объекты

На основе этих файлов с помощью специальных конструкторов создаются объекты. Каждый класс является по умолчанию наследником класса **Object**. Исключая примитивные типы, всё в Java является объектом

Пример создания ссылки на объект

```
Object myNewObject = new Object();
```

Где:

- **Object** — это тип ссылки **myNewObject**
- **myNewObject** — это имя ссылки на объект, по которой мы можем обращаться к созданному объекту
- **new** — специальный оператор, вызываемый для создания нового объекта из конструктора
- **Object()** — специальный метод класса **Object**, называемый конструктором и необходимый для создания объекта и выделения для него памяти

Тип данных String



2

Тип данных **String**

Строки — самый используемый класс практически для любого языка. В Java строки представлены классом **String**:

```
String message = "Hello world";
```

Также строку можно создать как объект через конструктор класса:

```
String message = new String("Hello world");
```

Строки можно складывать:

```
String s1 = "Hello";  
String s2 = "world";  
String result = s1 + " " + s2; // "Hello world"
```

Тип данных String

Каждая строка — это массив символов, у которого можно получить длину и каждый элемент:

```
String text = "Welcome to netology";  
char symbol = text.charAt(0); // W  
int textLength = text.length(); // 19
```

Сравнение строк:

```
String s1 = "Welcome to netology";  
String s2 = "Welcome to netology";  
  
// Неправильный способ сравнения строк  
System.out.println(s1 == s2); // сравниваются ссылки на строки  
// Правильный способ сравнения строк  
System.out.println(s1.equals(s2)); // сравниваются строки
```

`==` сравнивает ссылки на объекты, а они будут равны только в одном случае — если ссылка ведёт на один и тот же объект

Сложение строк

Допустимо складывать строки следующим образом:

```
String value1 = "The lesson";  
String value2 = "about";  
String value3 = "objects";  
String result = value1 + " " + value2 + " " + value3;
```

Однако так создаётся несколько дополнительных объектов в памяти JVM. Чтобы этого избежать, нужно использовать специальные классы:

```
StringBuilder result = new StringBuilder();  
result.append(value1);  
result.append(value2);  
result.append(value3);  
  
// Для получение результирующей строки нужно обязательно вызвать метод toString();  
result.toString();
```


Тип данных Enum



3

Тип данных Enum

Enum — это специальный тип перечисления именованных констант. Он позволяет решать типичные проблемы перечисления значений, а также вводит дополнительные проверки типов со стороны Java.

Константы могут иметь конструкторы, методы и переменные экземпляра.

Пример:

```
enum Season {  
    AUTUMN, WINTER, SPRING, SUMMER;  
}
```

```
System.out.println(Season.AUTUMN);
```

Обёртки над примитивами

Каждый примитивный тип имеет аналог ссылочного типа. Они называются «классы-оболочки» (wrappers):

```
short — Short  
byte — Byte  
int — Integer  
long — Long  
float — Float  
double — Double  
char — Character  
boolean — Boolean
```

Такие классы нужны для расширения функционала примитивных типов, например, для преобразования строки в число:

```
Integer.parseInt("1901");
```


Методы класса **Object**

- **void notify()** — возобновляет выполнение потока, которого ожидает вызывающий объект
- **void notifyAll()** — возобновляет выполнение всех потоков, которых ожидает вызывающий объект
- **String toString()** — возвращает строку, описывающую объект
- **void wait()** — бесконечно долгое время ожидает другой поток выполнения
- **void wait(long millis)** — определённое время ожидает другой поток выполнения
- **void wait(long millis, int nanos)** — определённое время ожидает другой поток выполнения